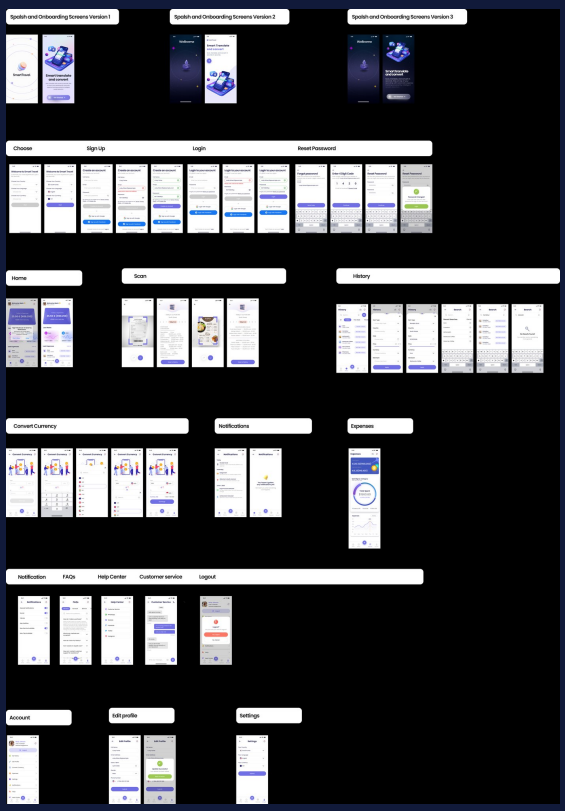
SMARTTRAVEL

Mobile Application API Implementation Guide

Screen-by-screen contracts for the SmartTravel mobile developer

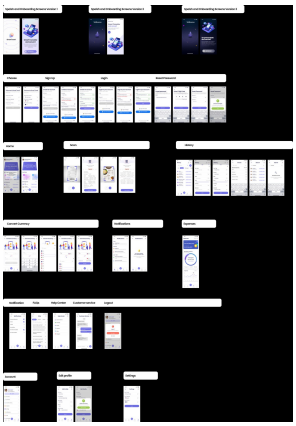


Backend: NestJS API | Authentication: Firebase | Database: SQL

Prepared from the supplied mobile design board and the implemented backend source - 14 August 2026

1. Design-to-API map

Use this as the build order and acceptance checklist.



Screen group	Primary API responsibilities
Splash / onboarding	Local-only UI. Load reference data before setup.
Choose	GET countries, currencies, languages; PATCH users/me after authentication.
Sign up / login / reset	Firebase-backed auth endpoints, refresh-token rotation.
Home	Profile, expense statistics, recent expenses, notifications.
Scan	Multipart receipt OCR, recommendation, expense creation.
History	Receipt list/detail and expense list with date/category filters.
Currency converter	Currency list plus conversion query.
Notifications	FCM device registration, list/read/delete, country-change check.
Expenses	Expense CRUD, categories, statistics.
FAQs / help / service	Public FAQs and authenticated support chat.
Account / edit / settings	GET/PATCH users/me and reference lists.
Interpreter / subscription	Audio conversation workflow and payment lifecycle.

Design note

The image contains alternative splash/onboarding versions. Choose one visual version; all three are local-only and use the same API behavior after the user taps Get Started.

2. Global API contract

Rules that apply to every mobile feature.

• Base URL: configure per environment; local default is http://localhost:3000. A physical phone cannot use localhost to reach the developer PC.

• Protected request header: Authorization: Bearer <Firebase idToken>.

• JSON requests use Content-Type: application/json. Uploads use multipart/form-data and must let the SDK generate the boundary.

• Firebase ID tokens expire after about one hour. Store the refresh token securely and call POST /auth/refresh before expiry or after a 401.

• Dates are ISO-8601 strings. Database identifiers are UUID strings. Currency/country/language selectors must store the returned database ID, not just the display code.

Successful response envelope

```
{
  "status_code": 200,
  "title": "Loaded successfully",
  "body": "Loaded successfully",
  "data": { ...endpoint-specific payload... }
}
```

Error envelope

```
{
  "status_code": 400,
  "title": "Human-readable error message",
  "body": "Human-readable error message",
  "data": null
}
```

Mobile client rule

Decode data only after checking both the HTTP status and status_code. A 204 delete has no body. Never display raw Firebase/provider error identifiers directly to users.

Common status handling

Status	Mobile behavior
200/201	Render data and update local cache.
204	Remove local row; do not parse JSON.
400	Show validation/business message.
401	Refresh once, retry once, then sign out.
403	Subscription/admin restriction.
404	Show not found and return safely.
502/503	Provider unavailable; offer retry.

3. Splash, onboarding and Choose

Screen behavior and reference-data contracts.



Splash and onboarding

- No backend call is required. Persist an onboardingComplete boolean locally.
- On Get Started, navigate to Choose for first-time users or Login for returning users.

GET /countries — Public

Populate Current Country. Optional Bearer token enables translated names.

```
[
  {
    "id": "country-uuid", "name": "Tunisia",
    "alpha2Code": "TN", "alpha3Code": "TUN", "flag": "...",
    "region": "Africa", "capital": "Tunis",
    "currencyId": "currency-uuid", "currency": { ... }, "isActive": true
  }
]
```

GET /currencies — Public

Populate Home Currency.

```
[{ "id": "currency-uuid", "code": "TND", "name": "Tunisian Dinar", "isActive": true }]
```

GET /languages — Public

Populate app and interpreter language selectors.

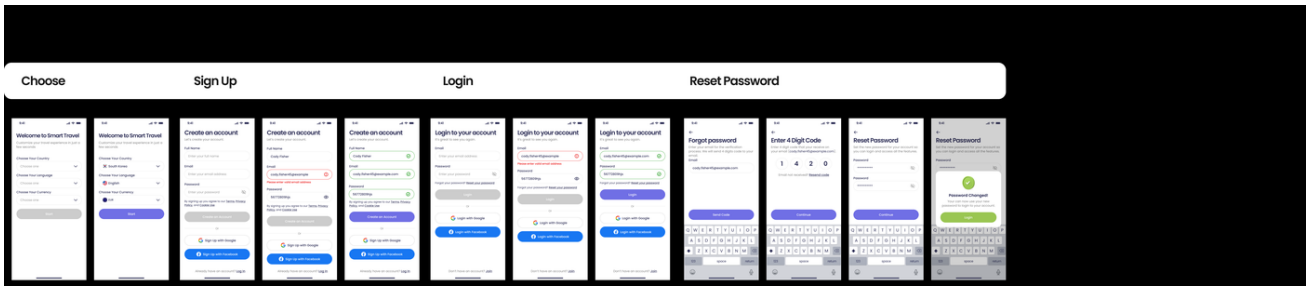
```
[{ "id": "language-uuid", "code": "ar", "name": "Arabic" }]
```

Setup sequencing

The current registration body accepts only email/password. Keep setup selections locally, register/login, then send them with PATCH /users/me as documented in Account & Settings.

4. Sign up and Login

Authentication screens from the supplied design.



POST /auth/register — Public

Create an email/password account.

Body

```
{ "email": "traveler@example.com", "password": "secret123" }
```

data

```
{
  "user": { "id": "user-uuid", "email": "traveler@example.com", ... },
  "idToken": "firebase-id-token",
  "refreshToken": "firebase-refresh-token"
}
```

POST /auth/login — Public

Email/password sign in. Body and response match registration.

POST /auth/oauth — Public

Google or Facebook sign in after obtaining a token from the native provider SDK.

```
{
  "provider": "google.com",
  "token": "provider-id-token-or-access-token"
}
```

Allowed provider values: google.com, facebook.com.

POST /auth/refresh — Public

Renew the Firebase ID token without asking the user to log in again.

```
Request:  { "refreshToken": "stored-refresh-token" }
data: {
  "access_token": "new-id-token",
  "refresh_token": "rotated-refresh-token",
  "expires_in": "3600", "token_type": "Bearer", "user_id": "..."
}
```

Secure storage

Store tokens in iOS Keychain / Android Keystore-backed secure storage. Do not store them in plain preferences. Replace both tokens when refresh returns a rotated refresh_token.

5. Reset password, session restore and logout

Three-step reset matching the email, OTP and new-password screens.

POST /auth/send-reset-mail — Public

```
{ "email": "traveler@example.com" }
```

Sends a four-digit OTP. OTP validity is five minutes.

POST /auth/validate-reset-mail — Public

```
Request: { "email": "traveler@example.com", "otp": "1234" }
data: { "resetPassToken": "5678" }
```

POST /auth/reset-password — Public

```
{
  "email": "traveler@example.com",
  "resetPassToken": "5678",
  "newPassword": "new-secret123"
}
```

Session restore

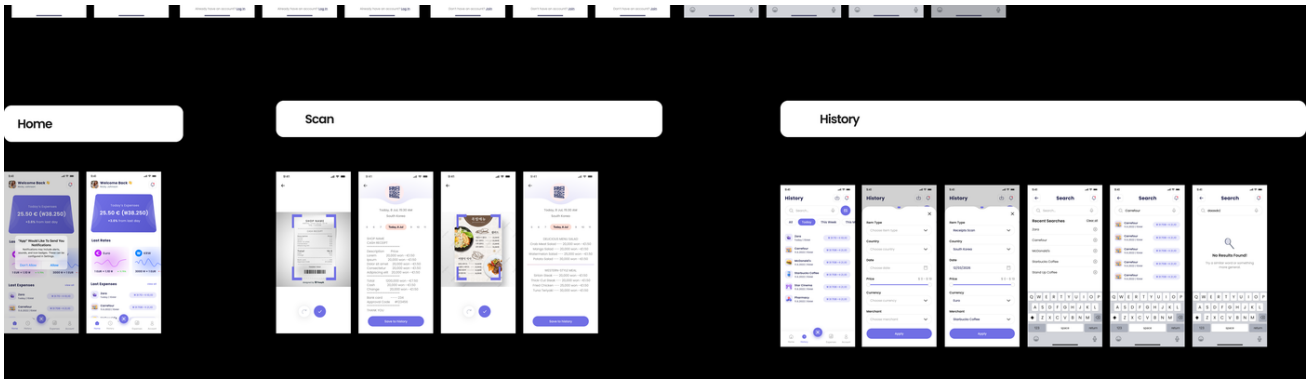
On app launch, load secure tokens, refresh if the ID token is near expiry, then call GET /users/me. Route to Login only when refresh fails.

Logout

There is no server logout endpoint. Delete the local ID/refresh tokens, unregister the push token with DELETE /notifications/devices/:token, clear cached private data, and return to Login.

6. Home dashboard

Data composition for the balance card, trends, rates and recent expenses.



GET /users/me — Bearer token

Profile, chosen country/currency/language and subscription state.

```
{
  "id": "user-uuid", "email": "traveler@example.com",
  "displayName": "Nadia Ali", "photoURL": "https://...",
  "currentCountryId": "...", "currentCountry": { ... },
  "currencyId": "...", "currency": { "code": "TND", ... },
  "languageId": "...", "language": { "code": "en", ... },
  "subscriptionStatus": "trial|active|expired|cancelled",
  "trialEndsAt": "ISO-date", "subscriptionEndsAt": "ISO-date|null"
}
```

GET /expenses/statistics?groupBy=month&startDate=...&endDate=... — Bearer token

Dashboard total, category chart and time series.

```
{
  "total": 1560.60, "count": 18, "currencyId": "home-currency-uuid",
  "groupBy": "month",
  "byCategory": [{ "categoryId": "...", "categoryName": "Food", "categoryColor": "#F59E0B", "total": 620, "count": 7 }],
  "byPeriod": [{ "period": "2026-08-01", "total": 420.50, "count": 5 }]
}
```

GET /expenses?startDate=...&endDate=... — Bearer token

Recent transactions; sort is newest first.

GET /notifications/me — Bearer token

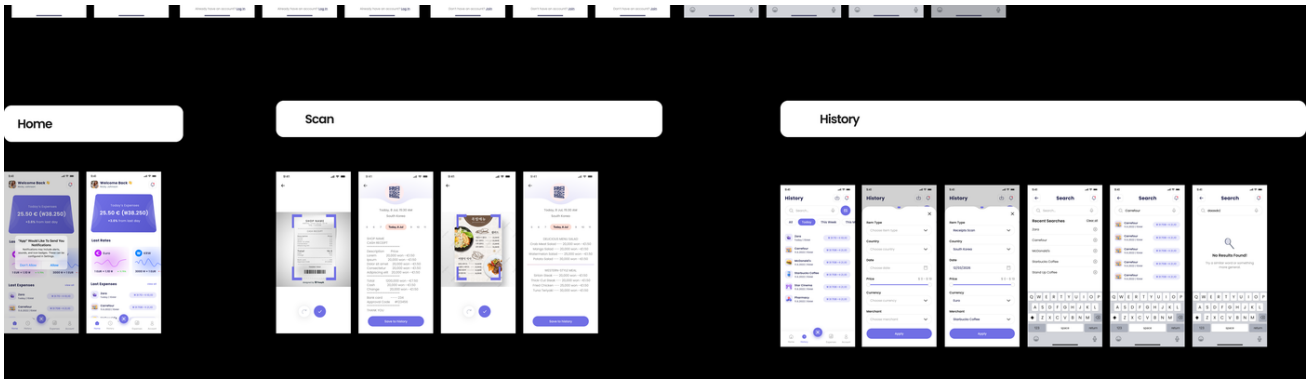
Unread badge and recent alerts.

Refresh behavior

Load these calls concurrently after GET /users/me succeeds. Pull-to-refresh repeats statistics, expenses and notifications. Do not block the whole home screen if one card fails.

7. Scan receipt and review result

Camera/gallery upload, OCR result and save-to-expenses flow.



POST /receipts/extract — Bearer token

Multipart OCR upload. The file field name must be exactly image.

```
Content-Type: multipart/form-data
image: <jpg|jpeg|png|webp|pdf file, maximum 10 MB>
```

data (representative receipt)

```
{
  "id": "receipt-uuid", "documentType": "receipt",
  "language": "fr", "country": "Tunisia", "merchant": "Carrefour",
  "address": "...", "phone": "...", "invoiceNumber": "...",
  "date": "2026-08-14", "time": "13:22", "currency": "TND",
  "subtotal": 30.0, "tax": 2.4, "total": 32.4,
  "paymentMethod": "card", "rawText": "...",
  "items": [{ "id": "...", "name": "Water", "quantity": 2, "unitPrice": 1.2, "totalPrice": 2.4 }],
  "convertedTotal": 32.4, "convertedCurrencyId": "home-currency-uuid"
}
```

POST /receipts/receipt-recommendations/:receiptId — Bearer token

Request category and description suggestions after OCR.

PATCH /receipts/:id — Public

Edit OCR fields before saving. Current backend route is not guarded; mobile should still send Bearer.

POST /expenses/from-receipt — Bearer token

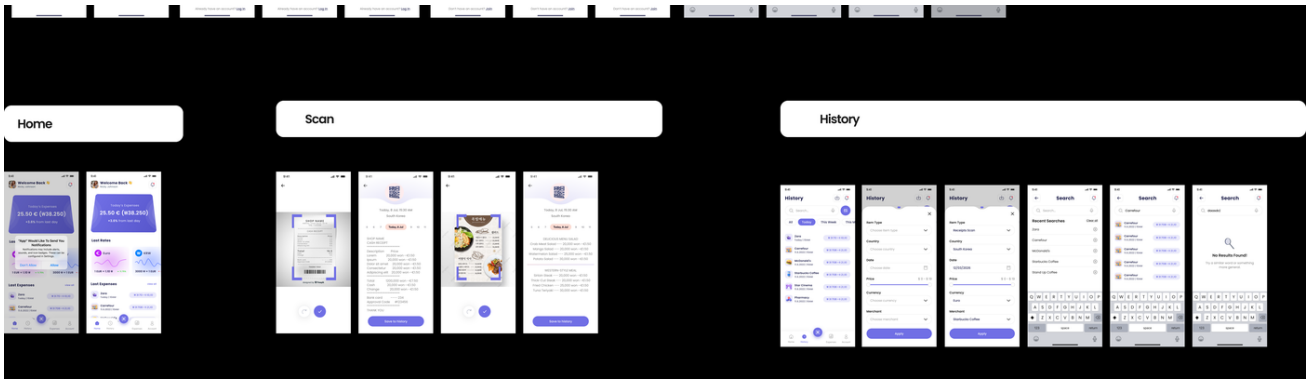
Create the expense when Save to History is tapped.

```
{
  "receiptId": "receipt-uuid", "categoryId": "category-uuid",
  "description": "Groceries", "amount": 32.4, "date": "2026-08-14",
  "merchantName": "Carrefour", "googlePlaceId": "optional",
  "merchantLatitude": 36.8, "merchantLongitude": 10.18
}
```

Important: the receipt recommendation may be absent or low-confidence. Keep category selection editable and require a categoryId before creating an expense.

8. History, search and filters

The design shows receipt cards plus country/currency/merchant/date filters.



GET /receipts	Bearer token

Fetch receipt history.

GET /receipts/:id	Bearer token

Open a receipt detail.

DELETE /receipts/:id	Bearer token

Delete a saved receipt.

GET /expenses?categoryId=:uuid&startDate=:iso&endDate=:iso	Bearer token

Filter expense history.

Backend gap to respect

The current receipt list controller accepts a query DTO but does not pass it into the service, and receipt rows are not user-owned in the entity. Therefore the country/currency/merchant search shown in the design is not safely supported server-side yet. For the current mobile build, filter the fetched list locally only for small datasets, and schedule backend pagination + per-user ownership before production.

Client-side search fields

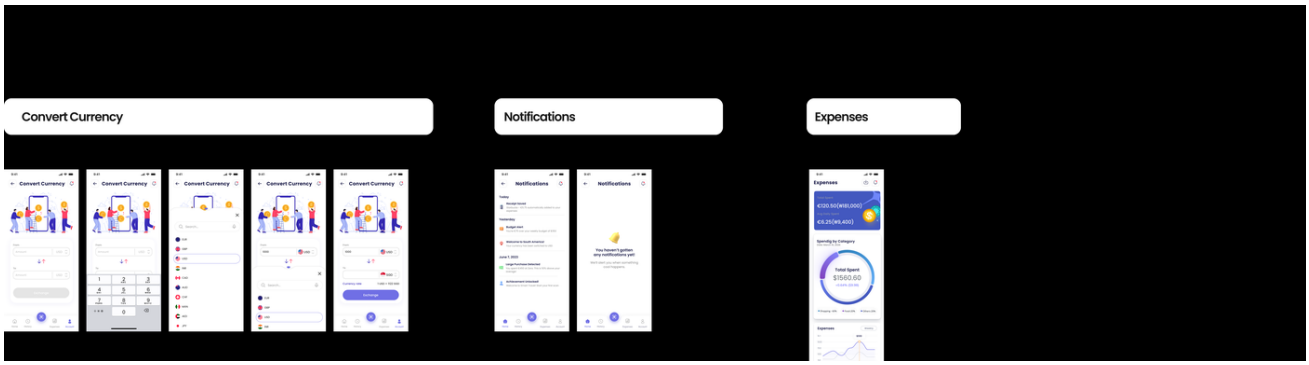
Filter	Receipt property
Country	country
Currency	currency
Merchant	merchant
Date	date
Amount	total

Empty state

Show No Results Found only after a successful fetch and filter. A network error should have its own Retry state.

9. Convert Currency

Currency selection, amount entry, swap and result.



GET /currencies — Public

Load and search active currencies. Retain both id and ISO code.

GET /currencies/convert?amount=100&fromCurrencyId=:uuid&toCurrencyId=:uuid — Public

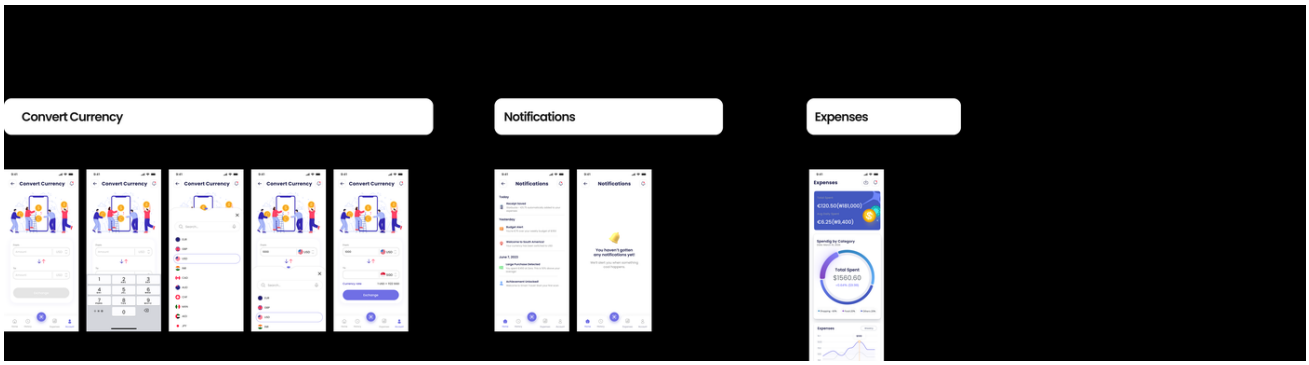
```
{
  "from": { "id": "...", "code": "USD", "name": "US Dollar" },
  "to": { "id": "...", "code": "EUR", "name": "Euro" },
  "originalAmount": 100,
  "convertedAmount": 85.71,
  "rate": 0.8571,
  "date": "2026-08-14"
}
```

Validation

amount must be at least 0.01. Both currency parameters are database UUIDs. Debounce conversion while typing; execute immediately on the Exchange button. Swap exchanges the selected IDs and runs again.

10. Expenses

List, add/edit, category chart and spending trend.



GET /expense-categories — Bearer token

```
[{ "id": "category-uuid", "name": "Food", "icon": "utensils", "color": "#F59E0B", "description": "..." }]
```

POST /expenses — Bearer token

```
{
  "amount": 42.5, "description": "Dinner", "shop": "Local Cafe",
  "date": "2026-08-14T19:30:00.000Z",
  "categoryId": "category-uuid", "currencyId": "currency-uuid",
  "googlePlaceId": "optional", "merchantAddress": "optional",
  "merchantLatitude": 36.8, "merchantLongitude": 10.18,
  "googleMapsUri": "https://...", "merchantMatchConfidence": 91
}
```

GET /expenses — Bearer token

Optional categoryId/startDate/endDate query parameters.

GET /expenses/:id — Bearer token

PATCH /expenses/:id — Bearer token

Partial body matching POST fields.

DELETE /expenses/:id — Bearer token

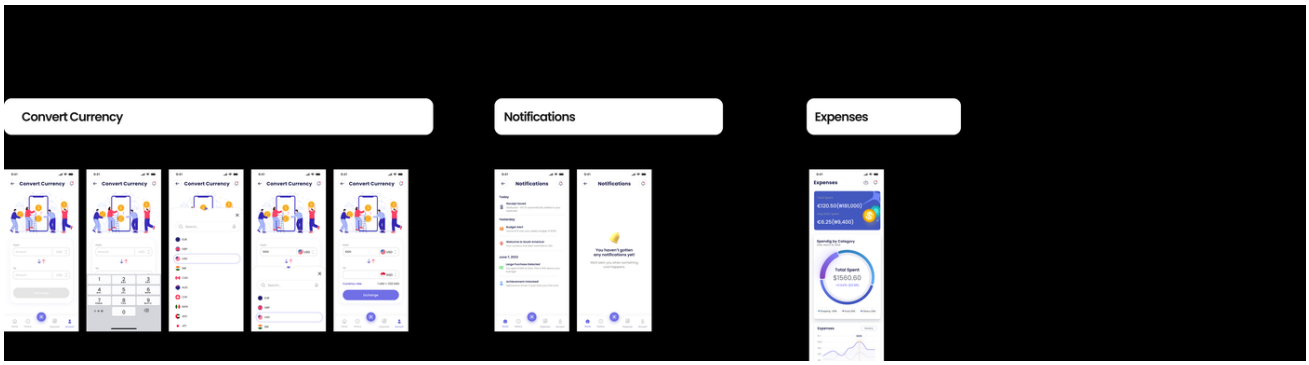
Returns HTTP 204 with no body.

Expense response detail

amount is returned as a decimal string. Each item also includes convertedAmount and convertedCurrencyId in the user's home currency. Parse monetary values with a decimal-safe strategy and format only for display.

11. Notifications and push registration

Inbox, empty state, unread badges and native push lifecycle.



POST /notifications/devices	Bearer token

Register the current FCM token after login and whenever it rotates.

```
{ "token": "fcm-device-token", "platform": "ios|android" }
```

GET /notifications/me	Bearer token

```
[{
  "id": "notification-uuid", "title": "Receipt extracted",
  "body": "Your receipt was extracted successfully",
  "data": { "type": "receipt.extracted", "receiptId": "..." },
  "source": "event", "read": false, "pushSent": true,
  "createdAt": "2026-08-14T13:25:00.000Z"
}]
```

PATCH /notifications/:id/read	Bearer token
PATCH /notifications/me/read-all	Bearer token
DELETE /notifications/:id	Bearer token
DELETE /notifications/devices/:token	Bearer token

Call on logout. URL-encode the token.

Notification tap routing

Route from data.type: receipt.* -> receipt detail; expense.* -> expense detail; payment.succeeded -> account/subscription; nearby-place -> maps URI; country_change_suggestion -> country confirmation dialog.

12. Location-driven notifications

Country-change prompt and hourly nearby recommendations.

POST /notifications/check-country — Bearer token

After native location permission, reverse-geocode coordinates to an ISO alpha-2 country code.

```
Request: { "detectedCountryCode": "FR" }
data: {
  "matches": false,
  "declaredCountry": { "id": "...", "name": "Tunisia", ... },
  "detectedCountry": { "id": "...", "name": "France", ... },
  "notification": { "id": "...", "data": {
    "type": "country_change_suggestion",
    "detectedCountryId": "...", "detectedCountryName": "France"
  }}
}
```

POST /explore/check-in — Bearer token

Nearby-place recommendation; call at most once per hour.

```
{
  "latitude": 48.8566, "longitude": 2.3522,
  "accuracy": 30, "languageCode": "en"
}
```

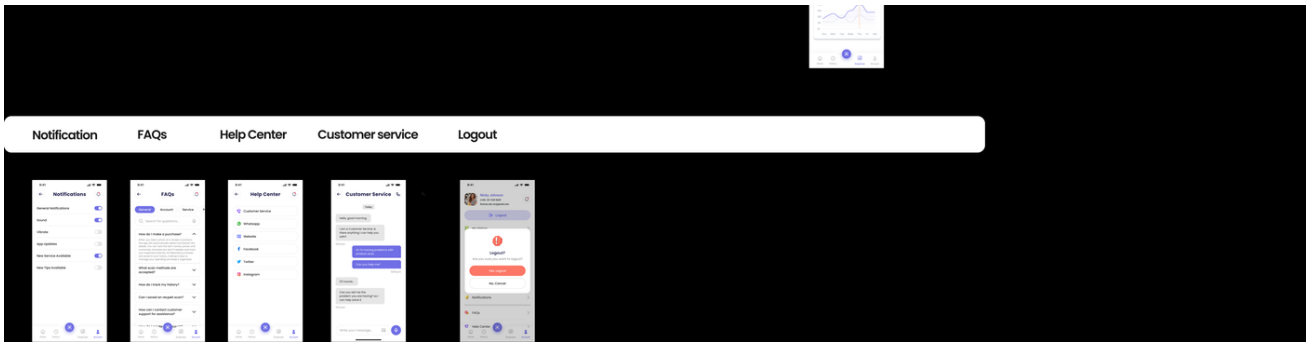
data: recommended boolean, optional notificationId, and place { id, name, address, googleMapsUri, distanceMeters }.

Permission behavior

Ask only in context, explain the benefit, and continue normally if denied. Country checks can run on login/foreground with a 12-hour client throttle. Nearby checks use a one-hour frontend and backend cooldown.

13. FAQs and Help Center

Public help content and static contact links.



GET /faqs/categories — Public

```
[{ "id": "category-uuid", "name": "Account", "description": "...", "order": 0, "isActive": true }]
```

GET /faqs?categoryId=:uuid — Public

List FAQs; omit categoryId for all active entries.

```
[{ "id": "faq-uuid", "question": "How do I reset my password?", "answer": "...", "categoryId": "...", "order": 0 }]
```

GET /faqs/category/:categoryId — Public

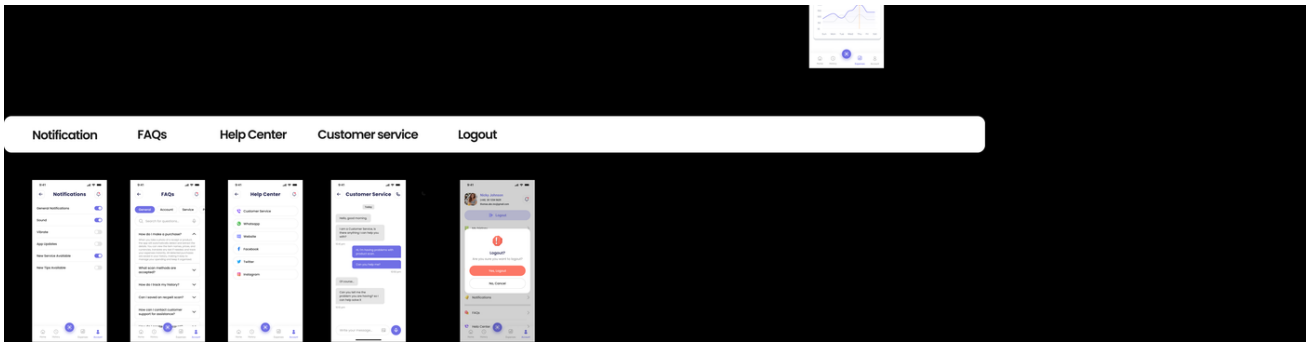
Alternative category-specific list.

Help Center contact buttons

WhatsApp, website, Facebook and Instagram in the design have no backend endpoint. Configure these URLs in mobile environment/remote config and open them with native deep links. Provide browser fallback when the target app is unavailable.

14. Customer Service chat

Conversation bootstrap, paginated history, sending and live updates.



GET /chat/conversations/me — Bearer token

Get or create the user's single support conversation.

```
{
  "id": "conversation-uuid", "userId": "user-uuid",
  "userDisplayName": "Nadia Ali", "lastMessagePreview": "Hello",
  "lastMessageAt": "ISO-date", "unreadByUser": 1, "unreadByAdmin": 0
}
```

GET /chat/conversations/:id/messages?before=:iso&limit=30 — Bearer token

```
[{ "id": "message-uuid", "conversationId": "...", "senderId": "...", "senderRole": "user|admin", "body": "Hello", "isRead": false, "createdAt": "IS
```

POST /chat/conversations/:id/messages — Bearer token

```
{ "body": "I need help with a scanned receipt." }
```

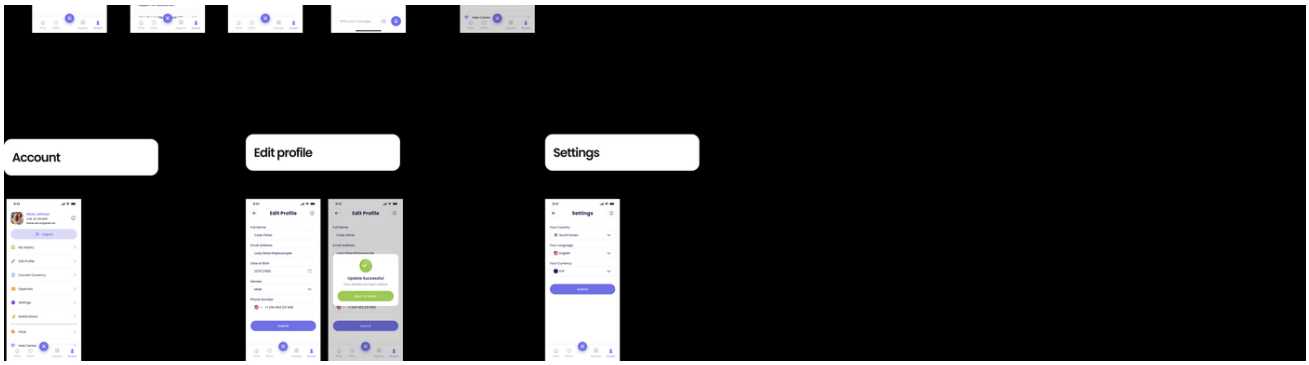
PATCH /chat/conversations/:id/read — Bearer token

Realtime

REST remains the source for writes and history. The backend mirrors chat to Firebase Realtime Database. Obtain a custom token with POST /auth/firebase-token, sign into Firebase, then subscribe to the conversation path described by the backend chat integration. Reconnect and re-fetch history after network loss.

15. Account, Edit Profile and Settings

Profile display, editable choices and app preferences.



GET /users/me	Bearer token
PATCH /users/me	Bearer token

Multipart form-data; submit only changed fields.

```
displayName: "Nadia Ali"              (optional string)
currencyId: "currency-uuid"             (optional UUID)
currentCountryId: "country-uuid"        (optional UUID)
languageId: "language-uuid"             (optional UUID)
image: <binary image>                    (optional; password accounts only)
```

Reference selectors

Use GET /countries, GET /currencies and GET /languages. After a successful update, replace the cached profile with response data and immediately update layout direction for RTL languages.

Settings boundary

Theme and most notification toggles shown in the design are currently local device settings. The backend supports device registration and notification records, but it does not persist per-topic preference switches. Do not pretend those switches sync across devices until a preferences endpoint is added.

16. Live Interpreter

Implemented feature not shown on the older design board; include it in the mobile build.

POST /interpreter/conversations — Bearer token

```
{ "travelerLanguage": "ar", "localLanguage": "en", "title": "Hotel check-in" }
```

GET /interpreter/conversations — Bearer token

GET /interpreter/conversations/:id — Bearer token

Returns the conversation with ordered turns.

POST /interpreter/conversations/:id/turns — Bearer token

Multipart audio upload, maximum 10 MB.

```
audio: <audio file>
speaker: "traveler" | "local"
sourceLanguage: "ar"
targetLanguage: "en"
```

Turn data

```
{
  "id": "turn-uuid", "speaker": "traveler",
  "sourceLanguage": "ar", "targetLanguage": "en",
  "sourceText": "...", "translatedText": "...",
  "sourceAudioUrl": "/files/interpreter/...",
  "translatedAudioUrl": "/files/interpreter/...",
  "sourceMimeType": "audio/webm", "translatedMimeType": "audio/mpeg"
}
```

DELETE /interpreter/conversations/:id — Bearer token

Audio client rules

Verify microphone permission and non-zero audio level before upload. Use a recorder format supported by the platform/backend. Build absolute playback URLs from the API base URL when the server returns /files/... paths. Disable duplicate submissions while STT/translation/TTS is running.

17. Subscription and payment

Plan selection, external checkout, reconciliation and profile refresh.

GET /payment-plans — Bearer token

Plans are converted to the user's selected currency.


```
[{ "id": "plan-uuid", "name": "Quarterly", "description": "...", "durationMonths": 3, "price": 29.99, "currency": { "id": "...", "code": "TND", "na
```


POST /payment/checkout/:planId — Bearer token


```
{ "paymentId": "payment-uuid", "checkoutUrl": "https://provider.example/..." }
```


POST /payment/:paymentId/verify — Bearer token

Poll every three seconds while checkout is visible and briefly after it closes.


```
{
  "id": "payment-uuid", "status": "pending|succeeded|failed",
  "provider": "konnect|moyasser", "amount": 29.99,
  "currency": { ... }, "plan": { ... }
}
```


Success flow

- Stop polling when status is succeeded or failed; impose a five-minute upper bound.
- On succeeded, call GET /users/me. The backend sets subscriptionStatus=active and subscriptionEndsAt based on plan duration.
- Use an in-app browser or provider SDK where possible. Do not infer success merely because the checkout window closed.

Environment requirement

Production APP_BASE_URL must be a public HTTPS backend URL so Konnect/Moyasar webhooks can reach it. Client verification is the recovery path, not a replacement for a correctly configured webhook.

18. Mobile implementation checklist

Definition of done and known backend boundaries.

Area	Acceptance criteria
Networking	Environment base URL; typed envelope decoder; timeout; one refresh retry; multipart support.
Security	Keychain/Keystore tokens; no secrets in logs; TLS in production; logout clears private cache.
Auth	Register/login/OAuth/reset; automatic refresh; GET users/me after restore.
Localization	Persist selected language; RTL layout; send Bearer on translatable public endpoints when available.
Offline/UX	Loading, empty, error and retry state for each screen; cancel stale requests.
Uploads	Validate type/size; show progress; prevent duplicate submit; preserve review state after failure.
Push	Permission prompt; FCM registration/rotation; unregister on logout; deep-link by data.type.
Payments	External checkout; provider verification; refresh profile; handle pending/failed/timeout.
Location	Contextual permission; reverse geocode; 12-hour country throttle; 1-hour nearby throttle.
QA	Test expired token, denied permissions, slow network, 502 AI errors, popup close and duplicate callbacks.

Known backend work before production scale

- Add receipt ownership by user, server-side history filtering and pagination.
- Guard receipt update/delete routes consistently.
- Add persisted notification preference endpoints if switches must sync across devices.
- Replace localhost URLs in stored file/profile URLs and payment callback configuration with environment-aware public URLs.
- Consider a dedicated mobile bootstrap endpoint only after profiling; parallel calls are acceptable initially.

Suggested mobile module boundaries

```
core/network       envelope, auth interceptor, refresh mutex
features/auth      login, register, reset, secure token store
features/profile   profile + reference selectors
features/scan      media capture, OCR upload, receipt review
features/expenses  CRUD, categories, statistics, history
features/notify    FCM, inbox, deep links, location checks
features/chat      REST history + Firebase realtime
features/interpreter recorder, upload, playback, conversation state
features/payment   plans, checkout, verification, entitlement refresh
```

19. Endpoint quick reference

Mobile-facing endpoints grouped by feature.

Feature	Endpoints
Auth	POST /auth/register; POST /auth/login; POST /auth/oauth; POST /auth/refresh; POST /auth/send-reset-mail; POST /auth/validate-reset-mail; POST /auth/reset-password; POST /auth/firebase-token
Profile/setup	GET/PATCH /users/me; GET /countries; GET /currencies; GET /languages
Currency	GET /currencies/convert
Receipts	POST /receipts/extract; POST /receipts/receipt-recommendations/:id; GET /receipts; GET/PATCH/DELETE /receipts/:id
Expenses	GET/POST /expenses; POST /expenses/from-receipt; GET /expenses/statistics; GET/PATCH/DELETE /expenses/:id; GET/POST /expense-categories
Notifications	POST /notifications/devices; GET /notifications/me; PATCH /notifications/:id/read; PATCH /notifications/me/read-all; DELETE /notifications/:id; POST /notifications/check-country; POST /explore/check-in
Help/chat	GET /faqs/categories; GET /faqs; GET /chat/conversations/me; GET/POST /chat/conversations/:id/messages; PATCH /chat/conversations/:id/read
Interpreter	GET/POST /interpreter/conversations; GET/DELETE /interpreter/conversations/:id; POST /interpreter/conversations/:id/turns
Payment	GET /payment-plans; POST /payment/checkout/:planId; POST /payment/:paymentId/verify

Handoff note

This guide describes the backend currently implemented in the repository at the preparation date. When the API changes, update the mobile typed models and this handoff document together. Swagger is exposed at /api-docs for interactive inspection.